

# MAAK – Summarizer Requirement

## Phase 1: File Understanding, Persistent Memory & Intelligent Summarization (Advanced Version)

---

### 1. System Objective (What We Are Building)

**MAAK – Summarizer** is designed to function like **ChatGPT with file memory**, not a simple one-time summarization tool.

The system must: - Accept user-uploaded files (PDF, DOCX, images) - Convert those files into machine-understandable text - Persist the extracted knowledge in a reusable form - Enable future summarization, querying, and retrieval **without re-uploading the file**

This requires a structured AI pipeline rather than a direct call to an LLM.

---

### 2. Non-Goals (What We Explicitly Avoid)

We are **not** building a naive workflow such as:

Upload → LLM → Summary → Forget

This approach fails because it: - Does not retain memory - Cannot support follow-up questions - Cannot perform semantic search - Does not scale with document size - Breaks the ChatGPT-like interaction model

Such a design only supports **single-use summarization** and is unsuitable for production systems.

---

### 3. Core Architecture (RAG-Based Design)

MAAK follows a **Retrieval-Augmented Generation (RAG)** architecture. In this model, the LLM is not the storage layer; it is only the reasoning engine.

#### End-to-End Flow

```
File Upload
↓
Text Extraction
↓
Text Chunking
↓
```

```
graph TD; A[Embedding Generation] --> B[Vector Storage (Persistent Memory)]; B --> C[Semantic Retrieval]; C --> D[LLM Reasoning (Summary / Answer)];
```

Each stage exists to solve a specific technical limitation of LLMs.

---

## 4. Detailed Pipeline Breakdown

### 4.1 File Upload ( /upload )

**Responsibility:** File persistence and identity assignment

- Accepts user files
- Stores the file (local or cloud)
- Records metadata (file ID, user ID, status)

**Key Principle:** This layer contains **no AI logic**. It is purely backend infrastructure.

---

### 4.2 Text Extraction ( /extract )

**Responsibility:** Convert files into raw text

LLMs do not process binary formats. All files must be transformed into plain text using: - PDF parsers - OCR engines - Multimodal LLMs (e.g., Gemini) used strictly as extractors

**Output:** - Unstructured, linear text

At this stage, the system has **data but no understanding**.

---

## 5. Chunking (Context Control Layer)

### Definition

Chunking is the process of splitting long extracted text into **smaller, semantically meaningful units**.

### Why Chunking Is Mandatory

- Prevents token-limit overflow
- Reduces contextual noise
- Enables fine-grained retrieval
- Improves response accuracy
- Minimizes inference cost

Without chunking, semantic retrieval becomes inaccurate and inefficient.

### Example

Original text (multi-paragraph):

```
Climate change affects agriculture by increasing temperatures.  
Farmers face water shortages.  
Crop yields are declining.
```

Chunked representation: - Chunk 1: Climate change affects agriculture by increasing temperatures. -  
Chunk 2: Farmers face water shortages. - Chunk 3: Crop yields are declining.

Each chunk is treated as an independent knowledge unit.

---

## 6. Embeddings & Vectorization (Semantic Encoding)

### What Is an Embedding?

An embedding is a high-dimensional numerical vector representing the **semantic meaning** of a text segment.

Text:

```
Climate change affects agriculture
```

Vector:

```
[0.12, 0.88, -0.09, ...]
```

### Why Embeddings Matter

- Enable meaning-based similarity search
- Remove dependency on keyword matching
- Allow scalable semantic retrieval

Embeddings are generated **per chunk**, not per document.

---

## 7. Vector Database (Persistent Knowledge Memory)

Each chunk and its embedding are stored in a vector-capable database.

**Stored fields typically include:** - file\_id - user\_id - chunk\_text - embedding vector

This database functions as the system's **long-term memory**, independent of the LLM.

---

## 8. Semantic Retrieval (Context Selection)

When a user requests a summary or asks a question:

1. The query is embedded into a vector
2. A similarity search is performed against stored embeddings
3. The most relevant chunks are selected
4. Only these chunks are passed to the LLM

This ensures: - High relevance - Minimal hallucination - Cost efficiency - Deterministic grounding in user-provided data

---

## 9. Summarization & Answer Generation (Reasoning Layer)

The LLM operates **only on retrieved context**, not on the full document.

This final step: - Applies reasoning - Generates summaries or answers - Does not store memory

Memory persistence is handled exclusively by the vector database.

---

## 10. Clear Separation of Responsibilities

| Stage      | Role                   |
|------------|------------------------|
| Extraction | File → Text            |
| Chunking   | Text → Context Units   |
| Embedding  | Context → Vectors      |
| Storage    | Persistent Memory      |
| Retrieval  | Context Selection      |
| LLM        | Reasoning & Generation |

Skipping any stage breaks the system's ability to scale or remember.

---

## 11. Phase 1 Completion Criteria

Phase 1 is considered complete when the system supports: - File upload and storage - Deterministic text extraction - Chunking logic - Embedding generation - Vector-based persistence - Retrieval-based summarization

---

## **Final Technical Insight**

LLMs are not databases. They are reasoning engines. Memory, retrieval, and scalability must be implemented explicitly.

Once this pipeline is understood, implementation becomes a straightforward engineering task.